Deep Closing: Enhancing Topological Connectivity in Medical Tubular Segmentation

Qian Wu, Yufei Chen, Wei Liu, Xiaodong Yue, and Xiahai Zhuang

***Abstract*— Accurately segmenting tubular structures, such as blood vessels or nerves, holds significant clinical implications across various medical applications. However, existing methods often exhibit limitations in achieving satisfactory topological performance, particularly in terms of preserving connectivity. To address this challenge, we propose a novel deep-learning approach, termed Deep Closing, inspired by the well-established classic closing operation. Deep Closing first leverages an AutoEncoder trained in the Masked Image Modeling (MIM) paradigm, enhanced with digital topology knowledge, to effectively learn the inherent shape prior of tubular structures and indicate potential disconnected regions. Subsequently, a Simple Components Erosion module is employed to generate topology-focused outcomes, which refines the preceding segmentation results, ensuring all the generated regions are topologically significant. To evaluate the efficacy of Deep Closing, we conduct comprehensive experiments on 4 datasets: DRIVE, CHASE_DB1, DCA1, and CREMI. The results demonstrate that our approach yields considerable improvements in topological performance compared with existing methods. Furthermore, Deep Closing exhibits the ability to generalize and transfer knowledge from external datasets, showcasing its robustness and adaptability. The code for this paper has been available at: https://github.com/5k5000/DeepClosing.**

***Index Terms*— Tubular structures, topology preserving, masked AutoEncoder, deep closing.**

I. Introduction

TUBULAR structures constitute significant anatomical features present in various medical images, encompassing entities such as blood vessels and airways. Despite the remarkable achievements of deep learning methodologies in medical image segmentation [1], [2], the segmentation of tubular structures remains a formidable endeavor, largely due to their intricate morphologies and diverse scales [3], [4].

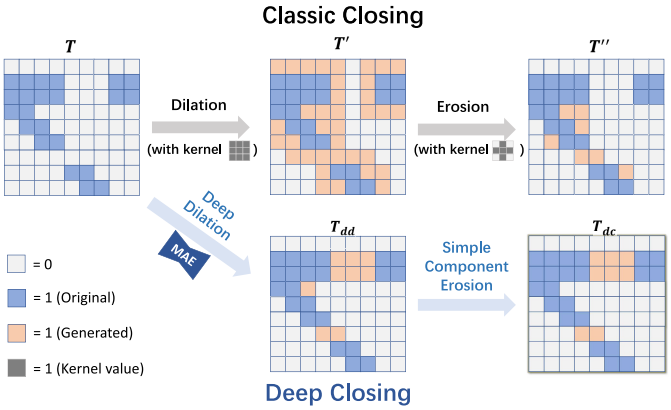


Fig. 1. An illustration of the Classic Closing Operation and the proposed Deep Closing. The traditional closing operation compromises two steps: Dilation and Erosion, which first generate new foreground points alongside borders and then eliminate part of them with predetermined kernels. Correspondingly, our approach introduces two enhanced constituents: Deep Dilation, which involves the indication of disconnection regions through the reconstruction process of a masked AutoEncoder (MAE) embedded with tubular shape prior; and Simple Component Erosion to filter out topologically insignificant regions.

It often encounters challenges in preserving continuity within diminutive and low signal-to-noise regions, thereby risking the disruption of structural topology. This issue predominantly emerges from the substantial intra-class variance apparent in pixel-level features within these localized regions.

The issue of connectivity defects in medical tubular structures is somewhat different from those in general segmentation tasks where the models typically aim to achieve better overlapping-based performance, i.e. higher Dice score. The disconnected regions tend to appear in thin and tiny parts which may have minimal effects on the overall overlapping-oriented optimization, thus usually causing the problem of disconnection under-focused. To surmount this challenge, some researchers have designed topology-aware methods [5], [6], [7], [8], [9], [10], [11] which typically impose extra topological constraints or networks designs during optimization. Some propose to enhance topological performance based on label refinement [12], [13], [14] which typically cascades multiple networks and makes the following ones serve as label error fixers for preceding ones. Besides, some general post-processing methods for segmentation like ConvCRF [15], CascadePSP [16] and Segrefiner [17], also show some effectiveness in mitigating the disconnection issue.

Manuscript received 9 April 2024; accepted 20 May 2024. Date of publication 27 May 2024; date of current version 29 October 2024. This work was supported in part by the National Natural Science Foundation of China under Grant 62173252 and in part by the Natural Science Foundation of Shanghai under Grant 21ZR1423900. *(Corresponding author: Yufei Chen.)*

Qian Wu, Yufei Chen, and Wei Liu are with the College of Electronic and Information Engineering, Tongji University, Shanghai 201804, China (e-mail: fivethousand@tongji.edu.cn; yufeichen@tongji.edu.cn; ldachuan@outlook.com).

Xiaodong Yue is with the Artificial Intelligence Institute, Shanghai University, Shanghai 200444, China (e-mail: yswantfly@shu.edu.cn).

Xiahai Zhuang is with the School of Data Science, Fudan University, Shanghai 200433, China (e-mail: zxh@fudan.edu.cn).

Digital Object Identifier 10.1109/TMI.2024.3405982

1558-254X © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on August 14,2026 at 07:34:55 UTC from IEEE Xplore. Restrictions apply.

The limitations of topology-aware methods, label refinement techniques, and post-processing strategies in medical image segmentation, particularly for tubular structures, present unique challenges. Topology-aware methods, despite incorporating topological constraints, are predominantly guided by the overlapping-based segmentation loss in an end-to-end training pattern. This dominance of segmentation loss means that any improvements in topological performance could be inherently limited, as the primary optimization direction remains focused on segmentation accuracy. Label refinement methods carry the risk of overfitting specific refinement criteria, limiting their generalizability to other models and making them primarily applicable only to the original models that they were trained with. Post-processing methods, while effective in the context of natural images, tend to underperform when applied to medical tubular datasets. This ineffectiveness is often due to the specialized nature of medical images, where the structural complexity and the critical importance of accurate topology present challenges that standard post-processing techniques cannot handle.

To address the above challenges, we aim to design a shape-oriented connectivity-enhancing method that is model-agnostic, generalizable, and can perform even when source images are unavailable. Following this purpose, we turn our attention to the classic closing operation, a technique specifically designed to bridge minor gaps within shapes. The Closing operation constitutes a foundational technique within image processing, assuming particular significance within the field of both medical image and natural image analysis. However, on the one hand, the judicious choice of the structuring elements like kernel size in the closing operation assumes paramount importance, as an improperly chosen size may result in insufficient gap closure or unintended amalgamation and smoothing of neighboring structures. On the other hand, though easy to implement and generalizable, the performance of this conventional technique is essentially limited.

Motivated by this, we propose a novel deep learning approach that extends the traditional closing operation, as shown in Fig.1. We replace the dilation process with the reconstruction of a designed Masked AutoEncoder, called *Deep Dilation*, and the erosion process with the proposed *Simple Component Erosion* operation. This approach leverages pre-trained knowledge to learn the intrinsic shape prior of tubular structures and focuses on the connectivity information to identify and refine the segmentation results.

Our contributions are summarized as follows: (1) As generally shown in Fig. 1, we propose a novel paradigm that revisits the classic closing operation with deep learning techniques to enhance connectivity in the segmentation of medical tubular structures. (2) We design a Simple Point Erosion module based on Digital Topology knowledge, which could efficiently deplete points that will not affect the topology under a paralleled pipeline, hereby identifying the topologically critical ones. This module could be integrated into the proposed framework and provide enhanced topological guidance. (3) Our extensive experiments demonstrate the effectiveness of improving topological performance and the transferability of our approach to learn from external datasets, enhancing overall segmentation performance.

II. Related Works

A. Topology-Aware Segmentation

Deep Learning methods have achieved huge success in medical segmentation [1], [18], [19], [20], [21], and most of them are optimized towards better overlap between prediction and target. However, in such a setting, topological coherence is often attached with less significance during optimization as it contributes little to the overall performance when measured with overlap-based metrics, such as Dice and IOU [9].

To improve the topological performance in tubular structures, some researchers propose adding topological constraints during optimization in order to guide models to focus on preserving topology. The core idea of these methods is to identify a group of topologically critical points and then impose an extra penalty on them. Hu et al. [5] first propose to utilize the persistence diagram to enforce the topology consistency between the prediction and ground truth. After that, they propose Discrete Morse Theory (DMT) Loss [6] which uses the Morse complex of the likelihood map to generate skeletons as topological constraints with time complexity of $O(nlog(n))$. They later simplify the critical points searching process with an efficient homotopy warping operation [7], theoretically achieving a time complexity of $O(n)$. However, it's worth noting that while this technique offers advantages, it may not always guarantee the identification of all topologically critical points and involves operations that can be computationally intensive on CPUs, such as depth distance transform and pixel-wise loops. The authors of [9] propose a novel, efficient, and differentiable skeletonization approach, called Centerline Dice (Cldice). They further introduce an additional loss on the computed centerlines of prediction and labels, in order to emphasize connectivity. However, their proposed skeletonization approach represents an approximation and may result in disconnected segments on centerlines. Stucki et al. [22] point out that the Wasserstein matching adopted in [5] could not always guarantee spatially correct matching and introduce a Betti Matching method to address such issue.

Beyond designing pixel-level topological constrains, Hu et al. [8] also pioneer another branch of methods in learning probabilistic topological representation, which could perform structure-level sampling, together with uncertainty, thus leading to better topological integrity. Gupta et al. [23] design a more fine-grained probabilistic structural representation learning by introducing a probabilistic DMT model, and further consider both inter-structure and intra-structure uncertainty, yielding more promising uncertainty maps.

PointScatter [10] designs a point prediction and regression paradigm for tubular structure segmentation, which could be utilized as an auxiliary task and embedded into existing methods, in order to enhance topological performance. DconnNet [11] decouples directional features from the shared subspace via enforcing the model to predict a connectivity map

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on August 14,2026 at 07:34:55 UTC from IEEE Xplore. Restrictions apply.

at an early stage, followed by an elaborately designed feature flow.

In this paper, we adopt the core concept of identifying topologically critical points for penalty. Compared with the aforementioned methods, we present a GPU-friendly, parallelized approach for identification, which not only guarantees the topology but also achieves a time complexity of $O(n)$, making it highly efficient and useful.

B. Enhancing Topological Performance via Label Refinement and Post-Processing

To enhance the topological performance, another approach is to conduct a secondary analysis of the pre-computed results using post-processing methods [24], [25], [26], [27]. These methods introduce additional topological constraints to the likelihood maps. In recent years, researchers have also adopted deep learning-based approaches to develop multi-stage or cascaded methods for improving topological performance. One such method, DoubleUnet [13], combines two UNet [1] structures and implicitly embeds the target of topology enhancement into the overall optimization process. Another approach, DVAE-refiner [12], proposes cascading a denoising variational Auto-Encoder (VAE) after a typical segmentation network. The VAE module first encodes precomputed shapes into a compact latent space and then reconstructs new ones to yield refined results. ErrorNet [14] cascades a noise-injected VAE to synthesize label errors and then trains another convolutional network to learn the synthetic error residue. During the inference stage, ErrorNet fuses segmentation results and error-corrected regions to yield better overall outcomes.

Beyond the aforementioned model-specific label refinement techniques, whose effectiveness is often contingent upon preceding models, several general post-processing strategies can also contribute to the enhancement of segmentation quality in tubular structures. These include morphological closing operations, Conditional Random Fields (CRF) [28], CascadePSP [16], and SegRefiner [17]. Notably, these approaches are not limited by model or class specificity, suggesting their broad applicability and potential in improving segmentation outcomes.

C. Masked Image Modeling & Denoising AutoEncoder

Recently, the autoregressive paradigm of masking and reconstruction has garnered significant attention from researchers in the field of computer vision. Notable contributions in this area include BEiT [29], MAE [30], and Simmim [31]. While these works primarily focus on pretraining Transformer architectures [32], [33], we have discovered that the technique of masking images can be adapted and modified to efficiently generate synthetic disconnected errors for tubular structures. Denoising AutoEncoders are neural networks that are trained to reconstruct clean data from noisy or corrupted inputs. By introducing a masking mechanism, where certain regions of the input are intentionally masked or corrupted, denoising AutoEncoders can learn to effectively recover the missing or corrupted information. This approach has shown promising results in various applications, including image denoising, inpainting, and now, in the generation of synthetic disconnected errors for tubular structures.

III. Method

A. Overview

Section III-B involves training an AutoEncoder under the mask image modeling paradigm with the designed pipeline and topological enhancement, which embeds the inherent shape prior of tubular structures into the AutoEncoder. In Section III-C, we design a topology enhancement module called Simple Points Erosion Module based on Digital Topology [34]. It could efficiently identify topologically critical points and powers several modules within our framework. In section III-D, we elaborate on how to combine the trained AutoEncoder in previous steps with the Simple Component Erosion (a derivative operation of the Simple Point Erosion module), to form our proposed pipeline, Deep Closing. The proposed framework is shown in Figure 2.

B. Masked Shape Reconstruction (Training Stage)

In this section, we present our proposed method to train an AutoEncoder via masked image (shape) modeling and boost it with the proposed topological constraints. After training, it could serve as a region of disconnection indicator.

1) Dynamic Mask Degradation:

a) Mask generator: This module has been developed with the objective of generating dynamic masks controlled by the hyperparameters p (patch size) and r (mask ratio). The process commences by creating a tensor filled with zeros, possessing the identical shape as the input binary label map L. Subsequently, this tensor is padded to the nearest multiple of the patch size. Following this, the padded tensor is partitioned into patches of dimensions $p_h \times p_w$, and a proportion r of these patches is converted into masks filled with ones, utilizing an efficient implementation proposed in [30]. Affine transformations, such as rotation and shearing, are subsequently applied to the binary mask. Finally, the resulting mask M is cropped to match the dimensions of L.

b) Recurrent tubeScaler: This module is designed for the objective of augmenting tubular structures in diversified scales efficiently. During training, we perform the proposed recurrent simple point erosion operation to the input L with randomly selected k_r:

$$L' = RSPE(L, \mathbf{0}, k_r) \tag{1}$$

where L' is the randomly scaled label map, and $\mathbf{0}$ represents an $h \times w$ zero map. The Recurrent Simple Point Erosion (RSPE) operation is introduced in Section III-C.3. A visualization of this process can be seen in Fig 2 (above the *Recurrent TubeScaler* module), demonstrating how L is progressively eroded while preserving the structural topology. Ultimately, the masked input L_M can be formulated as:

$$L_M = L' \odot (1 - M)$$

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on August 14,2026 at 07:34:55 UTC from IEEE Xplore. Restrictions apply.

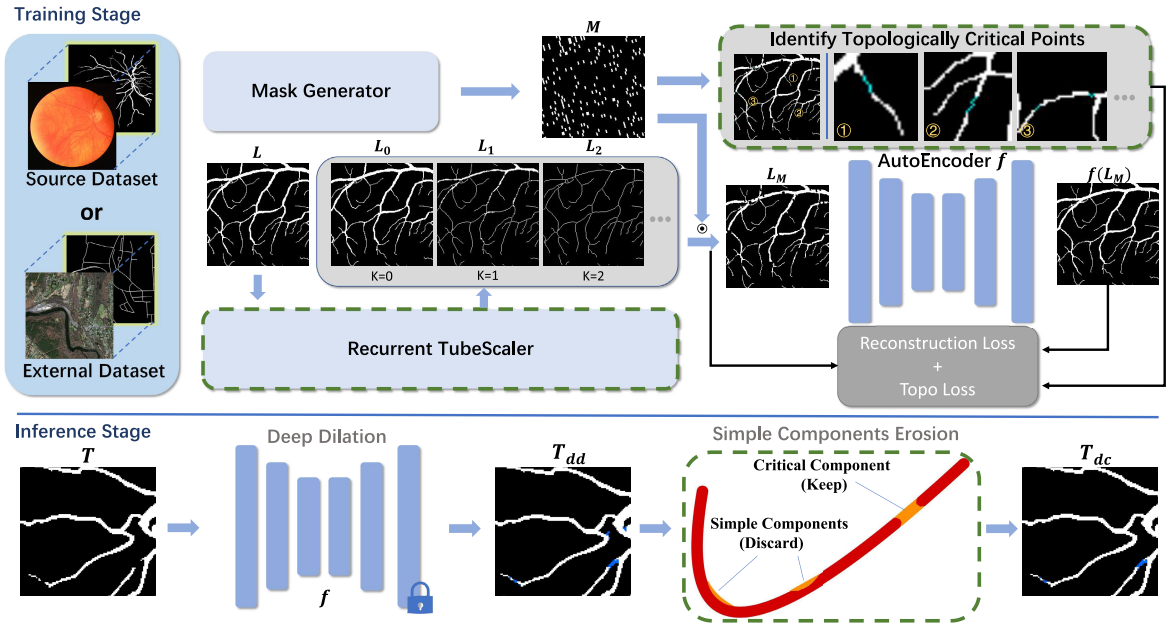


Fig. 2. Overview of the proposed methodology. During training, a convolutional AutoEncoder f learns inherent tubular shape prior using a masked image modeling paradigm. Shape annotations of tubular structures L, from internal or external datasets, drive the Recurrent TubeScaler module to generate structures across multiple scales, with subsequent masking to simulate disconnections. The AutoEncoder then reconstructs these masked shapes. After that, the reconstructed shapes are compared to the original shapes to compute loss, during which topologically critical points are identified to provide further topological constraints. In the inference phase, the trained AutoEncoder identifies potential disconnected structures (**Deep Dilation**) and refines them using **Simple Component Erosion**. Green dashed rectangles indicate modules employing the proposed Simple Points Erosion module.

2) Shape Reconstruction With Topological Constraints: The objective of the optimization is to enforce the AutoEncoder f to reconstruct intact tubular structures from the mask-corrupted ones, thus embedding shape prior of connectivity into the AutoEncoder. Following the setting in the MAE [30], we adopt Mean Square Error (MSE) as reconstruction loss $Loss_{rec}$:

$$Loss_{rec} = ||L' - \sigma(f(L_M))||_2^2 \tag{2}$$

where σ stands for the sigmoid function. Additionally, to encourage the prioritization of topologically critical structures by the function f, we incorporate an additional topological loss denoted as $Loss_{topo}$. This loss can be defined as follows:

$$Loss_{topo} = ||L' \odot M_{ns} - \sigma(f(L_M)) \odot M_{ns}||_2^2 \tag{3}$$

Here, $M_{ns} = (\lim_{k\to\infty} RSPE(L'; M; k)) \odot M$, indicating the non-simple points within the masked region M after undergoing complete recurrent simple points erosion. We present a visual demonstration of M_{ns} in Fig. 2 (above the AutoEncoder f), where the blue pixels in the structures stand for the M_{ns}. The comprehensive loss for our method is thus formulated as:

$$Loss_{total} = Loss_{rec} + \alpha Loss_{topo} \tag{4}$$

where α is a weight that controls the importance of topology loss.

C. Simple Points Erosion Module

In this section, we first introduce the basic concepts of Simple Points in Digital Topology. Then, we illustrate how we harness the properties of simple points to identify topologically critical points within the given structures.

Definition 1: (Simple Point [7], [34]). *A pixel in a binary image is considered a simple point if it can be inverted from foreground (FG) to background (BG) or vice versa without altering the topology of the image.*

Lemma 1: Determining whether a point is a 2D/3D simple point only requires analyzing its surrounding $3 \times 3(\times 3)$ *patch [7], [34].*

In this paper, we utilize the property of simple points to distinguish topologically significant points from others. Furthermore, it can be utilized as a method of thinning to generate a diverse range of tubular structures without altering the topology, serving as an efficient way of data augmentation.

Hu et al. [7] introduce an algorithm designed to gradually eliminate simple points within a structure. This approach is optimized using greedy strategies and theoretically reach a time complexity of $O(n)$, with n representing the number of pixels. However, there are certain considerations to keep in mind: (1) The nature of approximation employed by these strategies may present challenges when attempting to filter out all simple points. (2) Furthermore, the sequential computing aspect, including the need for connected component analysis for each pixel, could potentially result in computational inefficiency when integrated into deep learning frameworks.

In response to these challenges, we propose an efficient Simple Point classification method based on efficient enumeration and design a series of topological preserving operations. The detailed illustration is presented as follows and visualized in Fig. 3. The efficiency analysis and comparison are shown in Section IV-G.4.

1) Construct a Lookup Table of Simple Points: Let $D = \{(x_1, y_1), (x_2, y_2), \ldots, (x_{512}, y_{512})\}$ denote the simple point dataset, where each $x_i \in \{0, 1\}^{3\times3}$ represents a 3×3 binary

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on August 14,2026 at 07:34:55 UTC from IEEE Xplore. Restrictions apply.

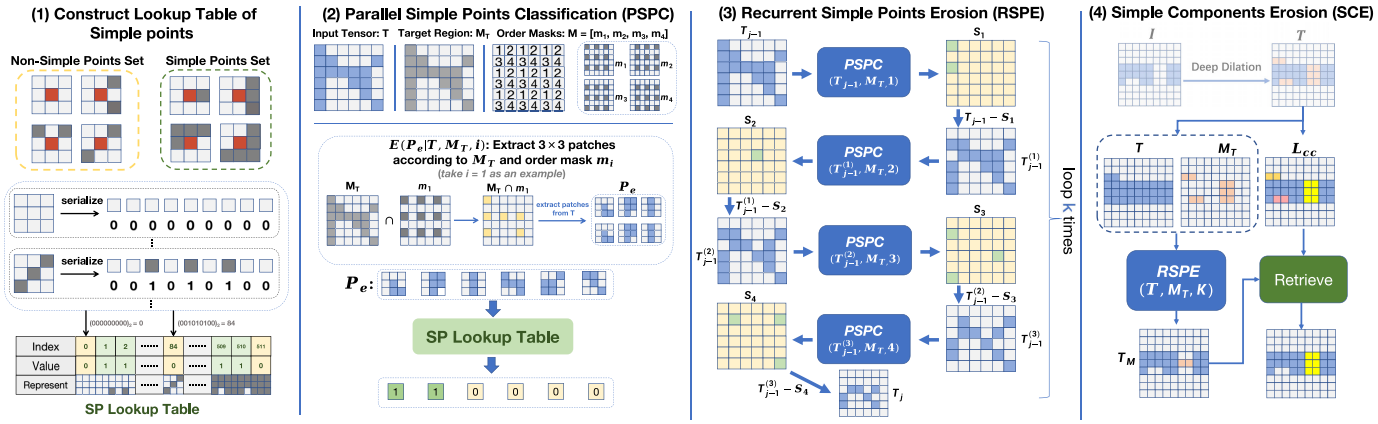


Fig. 3. An illustration of the proposed Simple Points Erosion module. (1) Construct a Lookup Table of Simple points (refer to section III-C.1). (2) Parallel Simple Points Classification (refer to section III-C.2). (3) Recurrent Simple Point Erosion (refer to section III-C.3). (4) Simple Component Erosion (refer to section III-C.4).

patch, and $y_i \in \{0, 1\}$ indicates the label that determines whether the central pixel of x_i is a simple point. The patches with $y = 0$ constitute the *Non-Simple Point Set*, and those with $y = 1$ constitute the *Simple Point Set*. Our goal is to construct a Simple Point Lookup Table ($SPLT$) that encapsulates D. To achieve this, we serialize each 3×3 binary matrix into a binary string, subsequently converting it into a unique decimal index. For example, the binary string $(001010100)_2$ is converted to 84 in decimal. This index acts as the locator within $SPLT$. Finally, we store the corresponding simple point label of each patch in $SPLT$. This procedure is illustrated in Figure 3 (1).

Let $P = (p_1, p_2, \ldots, p_m)$ represent a batch of 3×3 patches with size m to query, where $p_i \in \{0, 1\}^{3\times3}$, $\forall i \in \{1, \ldots, m\}$. The Simple Point Lookup Table ($SPLT$) can be applied to each patch in parallel and return their corresponding simple point tag: $S = SPLT(P), S = (s_1, s_2, \ldots, s_m)$, where S is a tuple of simple points labels with respect to P, and $\forall s_i \in S, (p_i, s_i) \in D$.

2) Parallel Simple Points Classification (PSPC): Utilizing the Simple Point Lookup Table ($SPLT$), our objective is to devise a function capable of simultaneously identifying simple points within a given structure $T \in \{0, 1\}^{H\times W}$, particularly within the designated region $M_T \in \{0, 1\}^{H\times W}$. This region, defined by M_T, specifies where to perform simple point classification, with H and W representing the height and width of the input structures, respectively. The value 1 in M_T delineates the area under consideration.

Furthermore, we construct a set of pre-defined binary masks within $PSPC$, denoted as $M = \{m_1, m_2, m_3, m_4\}$, where each mask m is defined by the following conditions:

$$\begin{cases} m_1(x, y) = (x \mod 2) \wedge (y \mod 2), \\ m_2(x, y) = (x \mod 2) \wedge \neg(y \mod 2), \\ m_3(x, y) = \neg(x \mod 2) \wedge (y \mod 2), \\ m_4(x, y) = \neg(x \mod 2) \wedge \neg(y \mod 2), \end{cases} \tag{5}$$

where $m(x, y)$ denotes the value at the x-th row and y-th column of mask m, these masks divide an image into four subregions, ensuring that for any foreground point within a mask, there are no other foreground points within its immediate 3×3 neighborhood. The significance of M in this context will be further discussed in the following section.

Prior to detailing the $PSPC$ workflow, we define a local patch extractor, $E(P_e|T, M_T, i)$, which extracts all 3×3 patches at the intersections defined by M_T and mask m_i, represented as $M_T \cap m_i$. It then returns a sequence of extracted patches, P_e. The workflow of $PSPC$ is delineated in Algorithm 1 and displayed in Figure 3 (2).

Algorithm 1 Parallel Simple Points Classification (PSPC)

Require:
$T \in \{0, 1\}^{H\times W}$: Input Structure,
$M_T \in \{0, 1\}^{H\times W}$: Specified Region,
i: the index of order mask m_i.

Ensure: Simple Point Label S: $S = \{s_j \mid s_j \in \{0, 1\}, j = 1, 2, 3 \ldots, |M_T \cap m_i|\}$

1: $P_e = E(T, M_T, i)$
2: $S = SPLT(P_e)$
3: **return** S

3) Recurrent Simple Points Erosion (RSPE): Guaranteed by the **Definition 1**, the depletion of any simple point will not affect the topology. However, the simultaneous depletion of all the simple points may not guarantee the consistency of the topology [7]. This is because simple points may be mutually relied on. For example, imagine a 4×4 binary image:

$$\begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

All the ones in the image can be classified as simple points individually, but the simultaneous depletion of all the points will lead to the alternation of topology. Hence, we need to decouple the mutual reliance. For this purpose, one could naively resort to sequential erosion of all the simple points (deplete every simple point one by one), but it suffers from low efficiency. Inspired by the **Lemma 1**, to perform a topology-preserving paralleled simple point erosion, we only need to guarantee that for each simple point to erode,

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on August 14,2026 at 07:34:55 UTC from IEEE Xplore. Restrictions apply.

no other point within its 3×3 surrounding patch will be eroded at the same time. To this end, we define four order masks m_1, m_2, m_3, m_4, to decouple the paralleled simple point erosion into 4 subiterations, and we perform $PSPC$ with only a region of m each time.

Moreover, as pointed out in [7], only the outmost points in the structures will be identified as simple points. And inner points could become simple points after their adjacent points are eroded. Hence, to completely erode all the simple points within the specified region, the simple point erosion algorithm is doomed to have a recurrent nature.

Hence, our designed operations Recurrent Simple Points Erosion is formally described in Algorithm 2, where *PSPC* represents the PSPC module illustrated in Section III-C.2, T represents the input tensor, T_0 is the initial value of T, and T_i is the updated value of T at iteration i. The mask M_T ensures that all modifications are confined to the specified region, where the mask has a value of 1. This allows for precise control over the areas of the tensor that are being modified.

Algorithm 2 Recurrent Simple Points Erosion (RSPE)

Require:
$T \in \{0, 1\}^{H \times W}$: Input structure,
$M_T \in \{0, 1\}^{H \times W}$: Specified region,
k: maximum iterations.
Ensure: T_k: eroded structure after k iterations
Initialize $T_0 = T$
for $j = 1$ **to** k **do**
  $S_1 = PSPC(T_{j-1}, M_T, 1)$
  $T_{j-1}^{(1)} = T_{j-1} - S_1$
  $S_2 = PSPC(T_{j-1}^{(1)}, M_T, 2)$
  $T_{j-1}^{(2)} = T_{j-1}^{(1)} - S_2$
  $S_3 = PSPC(T_{j-1}^{(2)}, M_T, 3)$
  $T_{j-1}^{(3)} = T_{j-1}^{(2)} - S_3$
  $S_4 = PSPC(T_{j-1}^{(3)}, M_T, 4)$
  $T_j = T_{j-1}^{(3)} - S_4$
end for
return T_k

After each iteration, simple points on the outermost edge of the structure T_i will be removed. The maximum iteration time k is a hyperparameter that determines the number of iterations performed. We define $RSPE(X; M_T; k)$ as the process of applying the proposed RSPE operation to the input binary tensor X within the specified region M_T for k iterations.

Proposition 1: For any arbitrary binary image T and any specified region M_T within T, and for any integer k, the Recurrent Simple Points Erosion operation, $RSPE(T, M_T, k)$, always preserves the topology of T.

Proof: For any given iteration within the k iterations, the Parallel Simple Points Classification (PSPC) is executed sequentially following the order of masks $m1, m2, m3$, and $m4$. For each PSPC application, due to the constraints imposed by mask m, each point classified as a simple point possesses a unique property: within its 3×3 neighborhood, it is the only simple point, ensuring that no other simple points within this neighborhood are eroded simultaneously. In other words, the 3×3 neighborhood state of any point undergoing parallel simple point classification is equivalent to the neighborhood state as if these points were eroded in a sequential order. Therefore, even if all simple points classified in a single PSPC application are eroded synchronously, the global topology remains unaffected. Given the recursive nature of RSPE, this property naturally generalizes to any arbitrary k. ■

Proposition 2: (Convergency) For any binary image T and a specified region M_T within T, the Recurrent Simple Points Erosion (RSPE) operation converges within a finite number of iterations k.

Proof: Consider the RSPE operation applied to a binary image T within the specified region M_T. The operation consists of iteratively applying Parallel Simple Points Classification (PSPC) using masks m_1, m_2, m_3, m_4 sequentially within each iteration.

1. **Non-Increasing Property**: In each iteration of RSPE, simple points within M_T may be eroded if identified. Since erosion removes points but never adds, the sequence of images produced by consecutive applications of RSPE is non-increasing in terms of the number of foreground points. This ensures that the process cannot oscillate or revert to a state with more foreground points than a previous state.

2. **Lower Bound**: The process of RSPE has a clear lower bound: the state where all erodible simple points within M_T have been eroded, leaving either no points or only non-simple points. This state is stable as no further erosion can occur.

3. **Termination**: Given the finite number of points in M_T and the non-increasing property of the RSPE process, the sequence of images will eventually reach the lower bound state. At this point, RSPE has converged, and further iterations will not alter the state of T within M_T.

Therefore, by the non-increasing property, the clear lower bound, and the finiteness of M_T, RSPE is guaranteed to converge within a finite number of iterations to a stable state where the binary image T no longer changes within the region M_T. ■

Note that even if we prove $RSPE$ could converge within finite iterations, it is not necessary to use it towards its convergence state for all scenarios. The intermediate results are also valuable for generating tubular structures of diverse scales, which we utilize for *Recurrent TubeScaler* in Section III-B.2. We provide an experimental study on how to select a proper iteration time for $RSPE$ in Section IV-G.5.

4) Simple Component Erosion (SCE): In this part, we study the component-level topology-preserving erosion operation. Following the Definition 1, we give the definition of a *Simple Component*.

Definition 2: (Simple Component) A component $C = \{v_1, v_2, v_3, v_4, \ldots, v_n\} (n \geq 1)$, which is a set of points in a binary image, is deemed a simple component if its addition or removal does not alter the overall topology of the image.

To ascertain whether a component is simple, one could naively resort to removing (adding) it and then check if the topological characteristics like connected components and loops have been affected. However, it is not computationally efficient as it may require performing whole-image-level

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on August 14,2026 at 07:34:55 UTC from IEEE Xplore. Restrictions apply.

analysis for each local component. To perform an efficient and simultaneous identification of all the specified components, we utilize the property of Simple Points to design the algorithm.

To first start with the identification of a single component C consisting of n points, let $M_C \in \{0,1\}^{H\times W}$ be the region mask of C, where $\sum_{v\in M_C} v = n$. We give the criterion to identify whether C is a simple component as proposition 3. Let T_C be the resultant structure of T after adequate simple point erosion, which could be defined as:

$$T_C = \lim_{k\to\infty} RSPE(T; M_C; k) \tag{6}$$

Proposition 3: C is a Simple Component $\iff \sum T_C \odot M_C = 0$.

Proof: We prove the proposition in both directions to establish the equivalence.

($\Rightarrow$) If C is a Simple Component:
Assume C is a simple component within the binary image T, but contrary to our proposition, assume $\sum T_C \odot M_C > 0$ after applying RSPE to convergence. This implies the existence of a set of points $A = \{a_1, a_2, \ldots, a_n\}$ $(n \geq 1)$ within the region M_C that remain uneroded. Let $Topo(x)$ be the topological characteristic of x.

By the nature of RSPE, which erodes only simple points, each point a_i in A must not be a simple point; otherwise, it would have been eroded by RSPE. According to the definition of a simple point (Definition 1), the removal of any a_i would alter the topology, implying $\text{Topo}(T_C \setminus A) \neq \text{Topo}(T_C)$.

Given that $T_C \setminus A$ represents the structure after the hypothetical removal of A from T_C, and considering $T \setminus C$ is equivalent to $T_C \setminus A$, we have $\text{Topo}(T \setminus C) = \text{Topo}(T_C \setminus A)$. From Proposition 2, we know $\text{Topo}(T) = \text{Topo}(T_C)$, leading to $\text{Topo}(T) \neq \text{Topo}(T \setminus C)$, which contradicts the premise that C is a simple component (as per Definition 2).

Hence, the assumption that $\sum T_C \odot M_C > 0$ must be false, and for C to be a simple component, $\sum T_C \odot M_C = 0$ must hold.

($\Leftarrow$) Only If $\sum T_C \odot M_C = 0$:
This means all the points within C have been eroded by $RSPE$. Guaranteed by Proposition 1 that the $RSPE$ operation will not change the topology, it exactly aligns with the Definition 2. Hence, C is proved a Simple Component. ■

Powered by the proposed RSPE operations, which erodes simple points globally and in parallel, we could discriminate simple components from non-simple ones simultaneously, according to the aforementioned Proposition 3.

As shown in Fig. 3(4), let $I \in \{0,1\}^{H\times W}$ be the input binary tensor to be processed, $T \in \{0,1\}^{H\times W}$ be the output of I after Deep Dilation. Then, we could have $M_T = T - I$ to be the generated potential regions of disconnection, which is the residue between T and I. We first assign a connected component label to each foreground pixel in M_T:

$$L_{cc}(x, y) = f_{cc}(x, y, M_T) \tag{7}$$

where f_{cc} represents the connected component function, x and y are the pixel position indices of M_T. $L_{cc} \in \mathbb{N}^{H\times W}$ stands for the connected component label map and $L_{cc}(x, y)$ is the label for the pixel at position (x, y). Then, we perform RSPE to T within the region M_T to obtain T_M:

$$T_M = \lim_{k\to\infty} RSPE(T; M_T; k) \tag{8}$$

We compute the remaining label in the region M_T as $L'_{cc} = M_T \odot T_M$, and then count all the distinct labels in L'_{cc}. In the end, for each remaining connected component label in L'_{cc}, we retrieve the component with the same value in M_T and add it to I iteratively, then we get the final result with all non-simple components preserved. (This process is referred to as **Retrieve** in Fig. 3)

D. Deep Closing: Deep Dilation + Simple Component Erosion (Inference Stage)

Our proposed framework, **Deep Closing**, for enhancing the connectivity in tubular structures, consists of two steps: Deep Dilation and Simple Component Erosion. Let f be the trained AutoEncoder in Section III-B, T be the pre-segmented binary map, we can utilize f to indicate potential regions of disconnection (the **Deep Dilation** process):

$$T_{dd} = \sigma'(f(T)) \tag{9}$$

where T_{dd} is the deep dilation result, and σ' means sigmoid activation together with thresholding operation. After that, the **Simple Component Erosion (SCE)** operation, which is illustrated in Section III-C.4, is applied to remove topologically unimportant regions:

$$T_{dc} = SCE(T_{dd}) \tag{10}$$

here T_{dc} is the **Deep Closing** result.

IV. Experiments and Results

A. Datasets

The **DRIVE** [35] dataset, sourced from a diabetic retinopathy screening program in the Netherlands, comprises 40 retinal images accompanied by annotations of retinal vessels. The images were captured using 8 bits per color plane at a resolution of 565 × 584 pixels. In line with the previous works [21], [36]. the dataset has been officially partitioned into a training set and a test set, each consisting of 20 images.

The **CHASE_DB1** [37] dataset encompasses 28 color retina images, each measuring 999 × 960 pixels. In accordance with prior research [11], the same data split for five-fold cross-validation is applied to evaluate our methods.

The **DCA1** [38] dataset consists of 134 X-ray coronary angiograms and their corresponding ground-truth images. The database is divided into two subsets: the training set consists of 100 images, and the remaining 34 angiograms comprise the test set.

The **CREMI**[1] dataset has three 3D neuron columns with resolution of 4 × 4 × 40 nm. For each column, we extract 125 2D slices and generate cell borders. Volumn A and Volumn B are used for training, and Volumn C is used as the test set.

[1]https://cremi.org/data/

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on August 14,2026 at 07:34:55 UTC from IEEE Xplore. Restrictions apply.

B. Evaluation Metrics

In line with and incorporating the evaluation metrics utilized by previous researchers [5], [6], [7], [9], [10], [11], [39], we have employed two types of metrics to assess the performance of our method. The first set of metrics consists of **overlapping-based metrics**, with some common measurements including Dice, Precision, and Recall. Additionally, we also compute an extra dice score between the predictions and ground truth, specifically after retaining the largest connected component, which we refer to as $Dice_l$.

The second set of metrics is the **topological metric**, which consists of the Centerline Dice (Cldice), Variation of Information (VOI), Adapted Rand Error (ARE), Betti Error (β_0, β_1), and Euler Characteristic Error (χ_{error}).

The Cldice [9] measures the dice similarity between skeletonized prediction and ground truth, thus putting more weight on topological performance.

The ARE[2] [40] measures the similarity between two clusters:

$$ARE = 1 - \frac{\sum_{ij} p_{ij}^2}{\alpha \sum_k s_k^2 + (1-\alpha)\sum_k t_k^2}$$

where p_{ij} is the probability that a pixel has the same label in the test image and in the true image, t_k is the the probability that a pixel has k in the true image, and s_k is the probability that a pixel has label k in the test image.

The VOI[1] [41] calculates the distance between two clusters based on mutual information:

$$VOI(X, Y) = H(X|Y) + H(Y|X)$$

where X is the ground-truth segmentation, then $H(X|Y)$ can be interpreted as the amount of under-segmentation and $H(Y|X)$ as the amount of over-segmentation.

The Betti Error[3] [5], [9], [10], [11] quantifies the local topological differences, with lower values indicating better topological accuracy. Here, the β_0 and β_1 correspond to the differences in the number of connected components and circular holes, respectively:

$$\beta_0 = \frac{1}{N}\sum_{i=1}^{N} |\mathrm{CC}(X_i) - \mathrm{CC}(Y_i)|$$

$$\beta_1 = \frac{1}{N}\sum_{i=1}^{N} |\mathrm{CH}(X_i) - \mathrm{CH}(Y_i)|$$

where CC and CH represent the 0-dimensional Betti number (number of connected components) and 1-dimensional Betti number (number of circular holes), respectively. X_i and Y_i represent the ith patch of the prediction and ground truth, respectively. N is the total number of patches.

Additionally, the χ_{error} metric measures the difference in Euler Characteristics between the prediction and ground truth. The Euler Characteristic χ [42] is computed as $\chi = V - E + F$, where V, E, and F denote the number of vertices, edges, and faces, respectively. The χ_{error} is also calculated by comparing the differences in Euler Characteristics between the patches in the prediction and ground truth:

$$\chi_{error} = \frac{1}{N}\sum_{i=1}^{N} |\chi(X_i) - \chi(Y_i)|$$

C. Implementation Details

Masked AutoEncoders are trained using a uniform distribution in the range [0.1, 0.2] to dynamically sample the mask rate and a uniform distribution in the range [2, 8] to sample the patch length. We adopt the UNet [1] as AutoEncoder f in our experiments. For the topological loss in Equation 4, the weight α is set to 0.1 for all experiments in this paper. The experiments are trained for 300 epochs using the AdamW optimizer with an initial learning rate of 1e-2 that decays to 1e-4 following a cosine annealing schedule. The networks are implemented using PyTorch [43] and MONAI [44].

For the computation of patch-based topological metrics, β_0, β_1, χ_{error}, we adopt the patch size of 64×64, in line with the Cldice [9] and PointScatter [10].

D. Comparing With the Segmentation Methods

We first construct a series of segmentation methods [1], [5], [7], [9], [10], [11], [12], [13], [14], [45] as benchmarks. Among these methods, Cldice [9], TopoLoss [5], WarpingLoss [7], PointScatter [10], DconnNet [11] are topology-aware segmentation methods, and DoubleUNet [13], DVAE_refiner [12], ErrorNet [14] are label-refinement based methods.

To assess the extent of improvement brought about by our proposed method, we employ it as a post-processing approach to enhance the performance of the aforementioned benchmarks. By integrating our method into the pipeline, we aim to demonstrate its ability to refine and enhance the segmentation results obtained from these existing techniques.

The quantitative results across the four datasets are presented in Table I. Furthermore, the qualitative analysis of these outcomes is depicted in Fig. 4. As evident from the data in Table I, a comprehensive summary of our findings indicates a consistent and substantial enhancement in the topological performance. This improvement is observed not only in terms of local connectivity, as demonstrated by metrics such as β_0, β_1, and X_{error}, but also in terms of the global completeness of the overall connecting structures, as exemplified by the $Dice_l$ metric.

The proposed method exhibits a marginal influence on overlapping-based metrics such as *Dice*, *Precision*, and *Recall*, with alterations observed at the order of $1e-3$. Notably, a subtle improvement in Recall alongside a slight reduction in *Precision* becomes discernible. This behavior originates from the fundamental aspect of our approach, which primarily identifies novel areas of disconnection, while not eliminating any pre-segmented regions.

E. Comparing With Post-Processing Methods

In the section IV-D, we conduct comparative analysis including label-refinement-based methods such as

[2]https://cremi.org/metrics/

[3]https://github.com/shizuo-kaji/CubicalRipser_3dim

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on August 14,2026 at 07:34:55 UTC from IEEE Xplore. Restrictions apply.

TABLE I

DEEP CLOSING RESULTS ON THE DRIVE, CHASE_DB, DCA AND CREMI DATASETS. THE PROPOSED METHOD IS EMPLOYED AS A POST-PROCESSING STEP FOR EXISTING SEGMENTATION APPROACHES. BETTER SCORES ARE INDICATED IN BOLD

method	Dice↑	Precision↑	Recall↑	ClDice↑	Dice_l ↑	ARE↓	VOI↓	β_0 ↓	β_1 ↓	Xerror↓
DRIVE										
UNet [1]	0.816 ± 0.016	**0.826 ± 0.037**	0.810 ± 0.049	0.819 ± 0.040	0.810 ± 0.019	0.236 ± 0.062	0.944 ± 0.151	0.792 ± 0.158	0.327 ± 0.131	1.068 ± 0.240
+ ours	0.816 ± 0.016	0.822 ± 0.037	**0.814 ± 0.048**	**0.820 ± 0.039**	**0.813 ± 0.018**	**0.220 ± 0.062**	**0.829 ± 0.148**	**0.509 ± 0.092**	**0.314 ± 0.134**	**0.756 ± 0.187**
DVAE_refiner [12]	0.794 ± 0.014	**0.752 ± 0.042**	0.847 ± 0.055	**0.820 ± 0.035**	0.791 ± 0.019	0.193 ± 0.079	0.842 ± 0.237	0.591 ± 0.129	0.322 ± 0.130	0.864 ± 0.211
+ ours	0.794 ± 0.014	0.750 ± 0.042	**0.849 ± 0.055**	0.819 ± 0.035	**0.793 ± 0.016**	**0.181 ± 0.072**	**0.754 ± 0.199**	**0.502 ± 0.098**	**0.308 ± 0.123**	**0.756 ± 0.185**
DoubleUnet [13]	0.822 ± 0.018	**0.820 ± 0.043**	0.830 ± 0.059	0.826 ± 0.036	0.819 ± 0.021	0.202 ± 0.076	0.784 ± 0.197	0.553 ± 0.106	0.259 ± 0.120	0.769 ± 0.182
+ ours	0.822 ± 0.018	0.818 ± 0.043	**0.831 ± 0.058**	0.826 ± 0.035	**0.820 ± 0.019**	**0.195 ± 0.071**	**0.737 ± 0.165**	**0.434 ± 0.066**	**0.256 ± 0.119**	**0.649 ± 0.151**
ErrorNet [14]	0.817 ± 0.015	**0.818 ± 0.040**	0.819 ± 0.050	0.816 ± 0.039	0.812 ± 0.018	0.217 ± 0.064	0.864 ± 0.156	0.854 ± 0.178	0.313 ± 0.127	1.115 ± 0.247
+ ours	0.817 ± 0.015	0.816 ± 0.039	**0.822 ± 0.049**	**0.817 ± 0.038**	**0.814 ± 0.016**	**0.208 ± 0.062**	**0.793 ± 0.150**	**0.596 ± 0.118**	**0.300 ± 0.125**	**0.838 ± 0.194**
TopoLoss [5]	0.816 ± 0.017	**0.830 ± 0.035**	0.807 ± 0.049	0.819 ± 0.038	0.811 ± 0.019	0.237 ± 0.063	0.928 ± 0.147	0.772 ± 0.144	0.322 ± 0.142	1.046 ± 0.227
+ ours	0.816 ± 0.017	0.827 ± 0.035	**0.809 ± 0.049**	0.819 ± 0.037	**0.814 ± 0.018**	**0.225 ± 0.062**	**0.845 ± 0.143**	**0.542 ± 0.102**	**0.304 ± 0.140**	**0.798 ± 0.193**
Cldice [9]	0.816 ± 0.015	**0.810 ± 0.038**	0.826 ± 0.049	0.828 ± 0.038	0.812 ± 0.016	0.214 ± 0.060	0.886 ± 0.149	0.706 ± 0.144	0.321 ± 0.129	0.967 ± 0.225
+ ours	0.816 ± 0.015	0.807 ± 0.039	**0.829 ± 0.048**	0.828 ± 0.038	**0.814 ± 0.016**	**0.202 ± 0.059**	**0.793 ± 0.139**	**0.489 ± 0.110**	**0.297 ± 0.136**	**0.730 ± 0.195**
LIOT [45]	0.814 ± 0.018	**0.817 ± 0.040**	0.814 ± 0.051	0.815 ± 0.043	0.807 ± 0.021	0.226 ± 0.075	0.901 ± 0.167	0.803 ± 0.163	0.331 ± 0.137	1.098 ± 0.263
+ Ours	0.814 ± 0.018	0.815 ± 0.039	**0.817 ± 0.050**	**0.816 ± 0.042**	**0.810 ± 0.019**	**0.214 ± 0.073**	**0.833 ± 0.159**	**0.523 ± 0.102**	**0.316 ± 0.136**	**0.792 ± 0.224**
Warping [7]	**0.817 ± 0.016**	**0.824 ± 0.036**	0.813 ± 0.047	0.819 ± 0.036	0.812 ± 0.018	0.228 ± 0.060	0.907 ± 0.147	1.061 ± 0.191	0.313 ± 0.131	1.322 ± 0.256
+ ours	0.816 ± 0.016	0.820 ± 0.037	**0.816 ± 0.047**	**0.820 ± 0.036**	**0.814 ± 0.017**	**0.217 ± 0.059**	**0.831 ± 0.146**	**0.643 ± 0.147**	**0.304 ± 0.129**	**0.889 ± 0.233**
Pointscatter [10]	**0.816 ± 0.016**	**0.803 ± 0.039**	0.834 ± 0.056	0.829 ± 0.037	0.810 ± 0.02	0.212 ± 0.075	0.922 ± 0.217	0.983 ± 0.169	0.321 ± 0.134	1.247 ± 0.265
+ ours	0.815 ± 0.015	0.799 ± 0.039	**0.838 ± 0.055**	**0.830 ± 0.036**	**0.814 ± 0.017**	**0.194 ± 0.069**	**0.785 ± 0.174**	**0.599 ± 0.104**	**0.286 ± 0.123**	**0.828 ± 0.204**
DconnNet [11]	0.806 ± 0.014	**0.790 ± 0.037**	0.825 ± 0.039	0.817 ± 0.037	0.802 ± 0.013	0.217 ± 0.046	0.914 ± 0.116	0.665 ± 0.103	0.348 ± 0.142	0.963 ± 0.210
+ours	0.806 ± 0.014	0.787 ± 0.037	**0.829 ± 0.039**	**0.819 ± 0.037**	**0.804 ± 0.014**	**0.200 ± 0.045**	**0.782 ± 0.104**	**0.416 ± 0.085**	**0.319 ± 0.135**	**0.679 ± 0.185**
CHASE_DB										
UNet	0.826 ± 0.023	**0.819 ± 0.056**	0.837 ± 0.034	0.841 ± 0.026	0.791 ± 0.039	0.268 ± 0.076	1.184 ± 0.259	0.400 ± 0.132	0.069 ± 0.021	0.454 ± 0.135
+ ours	0.826 ± 0.023	0.817 ± 0.055	**0.840 ± 0.033**	0.841 ± 0.026	**0.798 ± 0.041**	**0.245 ± 0.075**	**1.038 ± 0.243**	**0.326 ± 0.138**	**0.066 ± 0.021**	**0.377 ± 0.144**
DVAE_refiner	**0.806 ± 0.030**	**0.742 ± 0.059**	0.887 ± 0.027	**0.831 ± 0.028**	0.794 ± 0.037	0.203 ± 0.070	1.035 ± 0.304	0.352 ± 0.058	0.071 ± 0.022	0.410 ± 0.063
+ ours	0.805 ± 0.030	0.740 ± 0.059	**0.888 ± 0.027**	0.829 ± 0.029	**0.801 ± 0.032**	**0.175 ± 0.052**	**0.867 ± 0.225**	**0.325 ± 0.051**	**0.07 ± 0.02**	**0.378 ± 0.059**
DoubleUNet	0.825 ± 0.025	**0.821 ± 0.058**	0.835 ± 0.037	0.841 ± 0.028	0.792 ± 0.042	0.270 ± 0.075	1.197 ± 0.245	0.327 ± 0.087	0.071 ± 0.022	0.387 ± 0.097
+ ours	**0.826 ± 0.025**	0.818 ± 0.058	**0.837 ± 0.036**	**0.842 ± 0.028**	**0.798 ± 0.043**	**0.251 ± 0.071**	**1.069 ± 0.223**	**0.278 ± 0.069**	**0.065 ± 0.021**	**0.331 ± 0.079**
ErrorNet	0.814 ± 0.028	**0.766 ± 0.052**	0.872 ± 0.020	**0.837 ± 0.027**	0.787 ± 0.074	0.211 ± 0.075	1.019 ± 0.258	0.407 ± 0.102	**0.073 ± 0.024**	0.462 ± 0.104
+ ours	0.814 ± 0.028	0.763 ± 0.052	**0.875 ± 0.020**	0.836 ± 0.026	**0.804 ± 0.032**	**0.179 ± 0.052**	**0.832 ± 0.212**	**0.323 ± 0.080**	0.074 ± 0.023	**0.379 ± 0.085**
TopoLoss	0.829 ± 0.022	**0.826 ± 0.056**	0.836 ± 0.034	0.844 ± 0.025	0.784 ± 0.052	0.285 ± 0.091	1.252 ± 0.287	0.401 ± 0.062	0.068 ± 0.021	0.454 ± 0.065
+ ours	0.829 ± 0.022	0.823 ± 0.056	**0.839 ± 0.033**	0.844 ± 0.025	**0.793 ± 0.053**	**0.259 ± 0.090**	**1.099 ± 0.270**	**0.318 ± 0.060**	**0.067 ± 0.021**	**0.368 ± 0.064**
Cldice	0.827 ± 0.022	**0.818 ± 0.056**	0.841 ± 0.036	0.850 ± 0.025	0.789 ± 0.054	0.266 ± 0.096	1.158 ± 0.306	0.330 ± 0.064	0.066 ± 0.023	0.383 ± 0.070
+ ours	0.827 ± 0.022	0.816 ± 0.056	**0.844 ± 0.036**	0.850 ± 0.025	**0.793 ± 0.080**	**0.236 ± 0.096**	**0.970 ± 0.295**	**0.272 ± 0.058**	**0.063 ± 0.021**	**0.320 ± 0.063**
LIOT	0.819 ± 0.027	**0.813 ± 0.061**	0.829 ± 0.039	0.839 ± 0.027	0.788 ± 0.063	0.276 ± 0.083	1.242 ± 0.285	0.414 ± 0.117	0.072 ± 0.021	0.476 ± 0.136
+ Ours	0.819 ± 0.027	0.810 ± 0.060	**0.831 ± 0.038**	0.839 ± 0.027	**0.795 ± 0.067**	**0.251 ± 0.078**	**1.067 ± 0.269**	**0.336 ± 0.103**	**0.066 ± 0.022**	**0.384 ± 0.111**
Warping	0.830 ± 0.023	**0.822 ± 0.054**	0.843 ± 0.031	0.848 ± 0.023	0.798 ± 0.051	0.251 ± 0.085	1.099 ± 0.251	0.412 ± 0.075	0.068 ± 0.021	0.466 ± 0.072
+ ours	0.830 ± 0.023	0.819 ± 0.053	**0.846 ± 0.031**	**0.849 ± 0.023**	**0.803 ± 0.053**	**0.231 ± 0.087**	**0.967 ± 0.245**	**0.316 ± 0.060**	**0.064 ± 0.021**	**0.365 ± 0.062**
PointScatter	0.823 ± 0.025	**0.816 ± 0.056**	0.834 ± 0.036	0.837 ± 0.026	0.781 ± 0.041	0.288 ± 0.078	1.293 ± 0.279	0.457 ± 0.091	0.067 ± 0.020	0.511 ± 0.097
+ ours	0.823 ± 0.025	0.813 ± 0.056	**0.837 ± 0.035**	**0.839 ± 0.026**	**0.797 ± 0.041**	**0.246 ± 0.070**	**1.035 ± 0.240**	**0.298 ± 0.069**	**0.064 ± 0.022**	**0.350 ± 0.075**
DconnNet	0.818 ± 0.023	**0.799 ± 0.042**	0.841 ± 0.027	0.836 ± 0.028	0.800 ± 0.031	0.236 ± 0.055	1.058 ± 0.217	0.284 ± 0.070	0.067 ± 0.021	0.338 ± 0.076
+ ours	0.818 ± 0.023	0.797 ± 0.041	**0.843 ± 0.027**	0.836 ± 0.028	**0.805 ± 0.031**	**0.214 ± 0.056**	**0.913 ± 0.219**	**0.220 ± 0.052**	**0.066 ± 0.020**	**0.272 ± 0.061**
DCA										
UNet	0.784 ± 0.042	**0.822 ± 0.057**	0.756 ± 0.067	0.845 ± 0.046	0.761 ± 0.087	0.337 ± 0.140	1.065 ± 0.348	0.358 ± 0.174	0.068 ± 0.061	0.421 ± 0.183
+ ours	**0.785 ± 0.041**	0.815 ± 0.056	**0.763 ± 0.066**	**0.846 ± 0.046**	**0.783 ± 0.064**	**0.289 ± 0.113**	**0.884 ± 0.252**	**0.206 ± 0.122**	**0.058 ± 0.057**	**0.264 ± 0.136**
DVAE_refiner	0.791 ± 0.035	**0.742 ± 0.057**	0.853 ± 0.057	**0.857 ± 0.042**	0.792 ± 0.046	0.182 ± 0.096	0.691 ± 0.277	0.176 ± 0.091	0.065 ± 0.061	0.236 ± 0.100
+ ours	0.791 ± 0.034	0.740 ± 0.057	**0.855 ± 0.056**	0.856 ± 0.041	**0.797 ± 0.035**	**0.166 ± 0.073**	**0.630 ± 0.199**	**0.160 ± 0.086**	**0.058 ± 0.060**	**0.215 ± 0.095**
Doubleunet	0.789 ± 0.041	**0.846 ± 0.047**	0.745 ± 0.070	0.853 ± 0.048	0.767 ± 0.067	0.349 ± 0.123	1.076 ± 0.257	0.229 ± 0.096	0.060 ± 0.057	0.285 ± 0.133
+ ours	**0.790 ± 0.040**	0.841 ± 0.047	**0.750 ± 0.069**	**0.854 ± 0.048**	**0.780 ± 0.065**	**0.312 ± 0.119**	**0.926 ± 0.246**	**0.166 ± 0.085**	**0.054 ± 0.054**	**0.220 ± 0.114**
ErrorNet	**0.795 ± 0.036**	**0.736 ± 0.060**	0.871 ± 0.058	**0.855 ± 0.045**	0.801 ± 0.046	0.16 ± 0.10	0.636 ± 0.304	0.269 ± 0.125	0.064 ± 0.062	0.324 ± 0.159
+ ours	0.793 ± 0.035	0.732 ± 0.060	**0.874 ± 0.056**	0.852 ± 0.045	**0.803 ± 0.036**	**0.145 ± 0.077**	**0.574 ± 0.230**	**0.185 ± 0.077**	**0.061 ± 0.060**	**0.244 ± 0.106**
TopoLoss	**0.788 ± 0.038**	**0.752 ± 0.063**	0.834 ± 0.058	**0.849 ± 0.048**	0.793 ± 0.051	0.209 ± 0.096	0.763 ± 0.255	0.276 ± 0.106	0.059 ± 0.057	0.328 ± 0.145
+ ours	0.786 ± 0.039	0.747 ± 0.065	**0.837 ± 0.057**	0.845 ± 0.048	**0.796 ± 0.040**	**0.188 ± 0.078**	**0.681 ± 0.205**	**0.212 ± 0.087**	**0.056 ± 0.054**	**0.266 ± 0.120**
Cldice	**0.786 ± 0.039**	**0.759 ± 0.064**	0.821 ± 0.061	**0.850 ± 0.048**	0.781 ± 0.062	0.233 ± 0.105	0.826 ± 0.267	0.299 ± 0.142	0.061 ± 0.058	0.360 ± 0.175
+ ours	0.785 ± 0.040	0.754 ± 0.066	**0.825 ± 0.059**	0.849 ± 0.049	**0.792 ± 0.041**	**0.205 ± 0.082**	**0.724 ± 0.208**	**0.214 ± 0.097**	**0.064 ± 0.058**	**0.278 ± 0.132**
LIOT	**0.786 ± 0.043**	**0.813 ± 0.052**	0.768 ± 0.061	**0.851 ± 0.044**	0.774 ± 0.047	0.279 ± 0.121	0.889 ± 0.260	0.334 ± 0.117	0.063 ± 0.059	0.389 ± 0.124
+ Ours	0.784 ± 0.042	0.805 ± 0.052	**0.771 ± 0.060**	0.848 ± 0.044	**0.778 ± 0.034**	**0.256 ± 0.103**	**0.739 ± 0.207**	**0.227 ± 0.149**	**0.057 ± 0.054**	**0.268 ± 0.168**
Warping	**0.788 ± 0.039**	**0.747 ± 0.063**	0.839 ± 0.057	**0.848 ± 0.047**	**0.798 ± 0.042**	0.193 ± 0.080	0.715 ± 0.222	0.305 ± 0.158	0.058 ± 0.059	0.355 ± 0.183
+ ours	0.785 ± 0.040	0.740 ± 0.065	**0.843 ± 0.056**	0.843 ± 0.049	0.794 ± 0.042	**0.184 ± 0.078**	**0.678 ± 0.216**	**0.212 ± 0.116**	**0.054 ± 0.060**	**0.256 ± 0.140**
PointScatter	**0.781 ± 0.039**	**0.762 ± 0.052**	0.808 ± 0.072	**0.832 ± 0.053**	0.777 ± 0.063	0.262 ± 0.129	0.917 ± 0.337	0.338 ± 0.174	0.065 ± 0.053	0.398 ± 0.198
+ ours	0.780 ± 0.039	0.752 ± 0.055	**0.816 ± 0.068**	0.831 ± 0.052	**0.788 ± 0.042**	**0.217 ± 0.090**	**0.741 ± 0.220**	**0.176 ± 0.081**	**0.058 ± 0.054**	**0.229 ± 0.105**
DconnNet	**0.778 ± 0.039**	**0.680 ± 0.063**	0.916 ± 0.034	**0.872 ± 0.041**	0.776 ± 0.043	0.125 ± 0.075	0.607 ± 0.328	0.249 ± 0.151	0.074 ± 0.064	0.316 ± 0.178
+ ours	0.776 ± 0.040	0.675 ± 0.064	**0.919 ± 0.033**	0.867 ± 0.045	**0.779 ± 0.040**	**0.094 ± 0.047**	**0.446 ± 0.191**	**0.168 ± 0.103**	**0.069 ± 0.058**	**0.226 ± 0.114**
CREMI										
UNet	0.914 ± 0.015	**0.945 ± 0.012**	0.885 ± 0.018	0.968 ± 0.011	0.904 ± 0.047	0.141 ± 0.064	0.619 ± 0.206	0.124 ± 0.035	0.138 ± 0.026	0.251 ± 0.048
+ ours	0.914 ± 0.015	0.943 ± 0.013	**0.887 ± 0.018**	0.968 ± 0.011	**0.910 ± 0.031**	**0.129 ± 0.049**	**0.560 ± 0.161**	**0.093 ± 0.024**	**0.132 ± 0.026**	**0.215 ± 0.037**
DVAE_refiner	0.917 ± 0.014	**0.884 ± 0.016**	0.953 ± 0.014	0.969 ± 0.010	0.911 ± 0.028	0.063 ± 0.044	0.280 ± 0.095	0.098 ± 0.026	**0.142 ± 0.027**	0.23 ± 0.04
+ ours	0.917 ± 0.014	0.883 ± 0.016	0.953 ± 0.014	0.969 ± 0.010	0.911 ± 0.028	**0.062 ± 0.044**	**0.269 ± 0.090**	**0.080 ± 0.018**	0.143 ± 0.027	**0.214 ± 0.035**
DoubleUnet	0.931 ± 0.018	**0.927 ± 0.012**	0.935 ± 0.025	0.966 ± 0.013	0.925 ± 0.035	0.079 ± 0.059	0.362 ± 0.179	0.090 ± 0.028	0.140 ± 0.027	0.219 ± 0.043
+ ours	0.931 ± 0.018	0.926 ± 0.012	**0.936 ± 0.025**	**0.967 ± 0.012**	**0.927 ± 0.032**	**0.073 ± 0.053**	**0.332 ± 0.151**	**0.073 ± 0.021**	**0.138 ± 0.026**	**0.202 ± 0.036**
ErrorNet	0.921 ± 0.015	**0.897 ± 0.015**	0.947 ± 0.016	0.968 ± 0.011	0.916 ± 0.027	0.071 ± 0.044	0.327 ± 0.125	0.118 ± 0.033	0.141 ± 0.027	0.248 ± 0.044
+ ours	0.921 ± 0.015	0.895 ± 0.015	**0.948 ± 0.016**	0.968 ± 0.011	0.916 ± 0.027	**0.068 ± 0.042**	**0.307 ± 0.104**	**0.089 ± 0.022**	0.141 ± 0.028	**0.221 ± 0.037**
TopoLoss	**0.920 ± 0.016**	**0.903 ± 0.018**	0.938 ± 0.015	0.959 ± 0.013	**0.919 ± 0.027**	0.078 ± 0.042	0.365 ± 0.119	0.281 ± 0.063	0.150 ± 0.026	0.406 ± 0.069
+ ours	0.919 ± 0.017	0.899 ± 0.020	**0.940 ± 0.015**	0.959 ± 0.013	0.917 ± 0.028	**0.077 ± 0.043**	**0.344 ± 0.104**	**0.210 ± 0.038**	0.150 ± 0.027	**0.341 ± 0.049**
Cldice	**0.904 ± 0.018**	**0.884 ± 0.023**	0.925 ± 0.015	**0.955 ± 0.014**	0.909 ± 0.029	0.094 ± 0.044	0.426 ± 0.124	0.174 ± 0.047	**0.147 ± 0.028**	0.307 ± 0.056
+ ours	0.903 ± 0.018	0.881 ± 0.023	**0.926 ± 0.015**	0.954 ± 0.014	0.909 ± 0.029	**0.092 ± 0.043**	**0.406 ± 0.113**	**0.143 ± 0.032**	0.151 ± 0.029	**0.278 ± 0.044**
LIOT	0.910 ± 0.014	**0.922 ± 0.019**	0.903 ± 0.021	0.957 ± 0.015	0.908 ± 0.031	0.103 ± 0.045	0.424 ± 0.134	0.169 ± 0.055	0.153 ± 0.029	0.288 ± 0.059
+ Ours	0.910 ± 0.014	0.920 ± 0.019	**0.904 ± 0.021**	**0.958 ± 0.014**	**0.911 ± 0.030**	**0.098 ± 0.046**	**0.391 ± 0.117**	**0.123 ± 0.048**	**0.149 ± 0.029**	**0.247 ± 0.048**
Warping	0.921 ± 0.014	**0.916 ± 0.014**	0.927 ± 0.017	0.969 ± 0.010	0.915 ± 0.029	0.092 ± 0.046	0.428 ± 0.143	0.125 ± 0.037	0.138 ± 0.025	0.251 ± 0.047
+ ours	0.921 ± 0.014	0.914 ± 0.014	**0.928 ± 0.016**	**0.970 ± 0.011**	0.915 ± 0.030	**0.089 ± 0.047**	**0.395 ± 0.127**	**0.087 ± 0.025**	**0.137 ± 0.025**	**0.212 ± 0.035**

DVAE_refiner [12], DoubleUNet [13], and ErrorNet [14], which could be recognized as End-to-End post-processing approaches to some extent. In this part, we extend to include a range of pure ad-hoc post-processing methods. This broadened analysis encompasses the **classic Closing operation**, **ConvCRF** [15] (a conditional random field-based approach), and two deep learning-based methods for general image segmentation: **CascadePSP** [16] and **SegRefiner** [17]. Detailed quantitative and qualitative results of this expanded comparison are presented in Table II and Figure 5.

Our results reveal that the kernel size of the classic closing operation (dilation followed by erosion) and ConvCRF critically impacts performance. Excessively large kernels lead to over-smoothing and loss of detail, while overly small kernels are insufficient for bridging discon-

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on August 14,2026 at 07:34:55 UTC from IEEE Xplore. Restrictions apply.

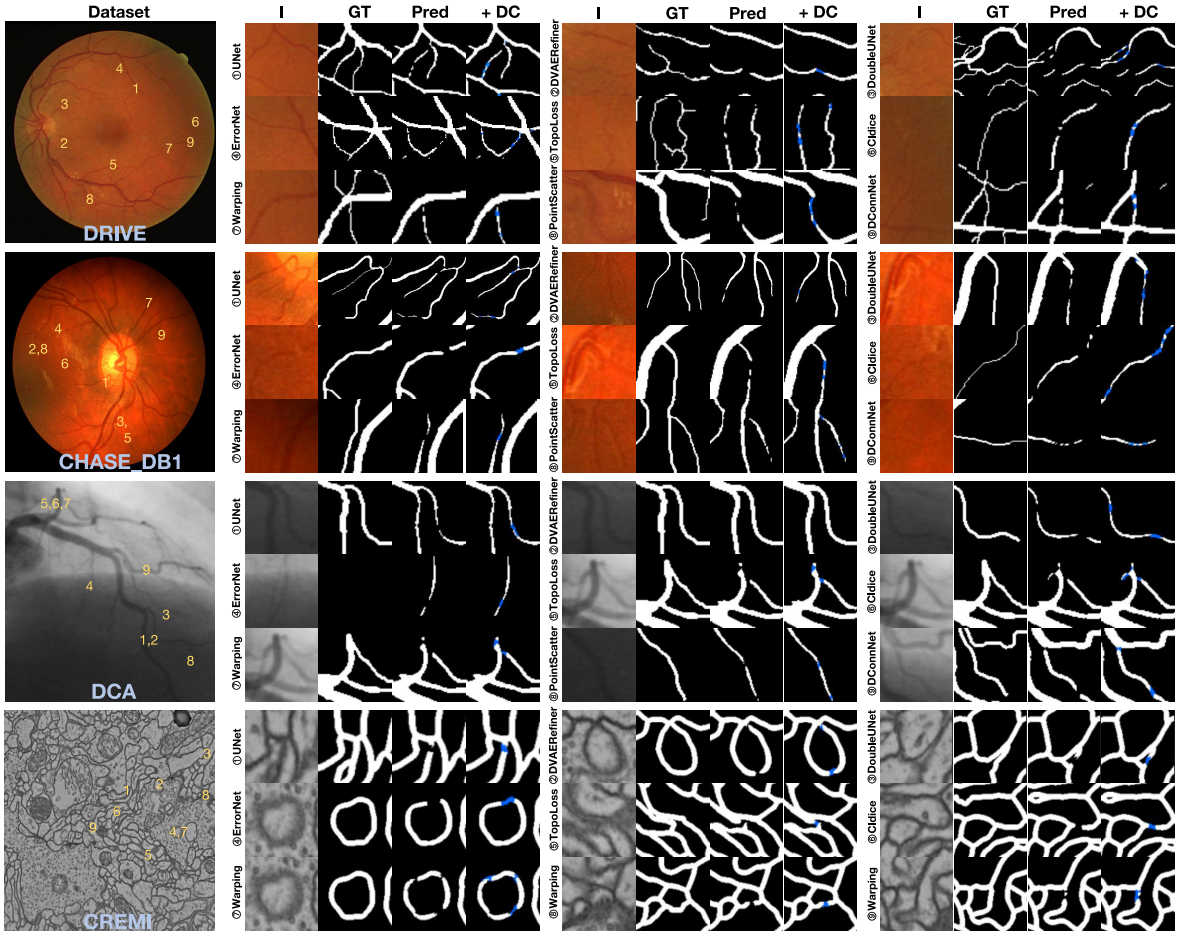


Fig. 4. Visualization of the proposed method on 4 public datasets. For each sample from different datasets, nine representative patches predicted by different existing methods, together with the post-processed results by the proposed approach, are shown. I, GT, Pred, and DC correspond to Image patch, GroundTruth, Prediction, and Deep Closing, respectively. Note that the position indices on the dataset images are used to identify the extracted patches for visualization, corresponding to methods with the same indices. Blue pixels in the '+DC' column represent the regions of disconnection indicated by our method.

TABLE II

COMPARISON WITH POST PROCESSING METHODS ON THE DRIVE DATASET. POSTPROCESSING METHODS(CLOSING, CONVCRF [15], CASCADEPSP [16], SEGREFINER [17] AND OURS) ARE EMPLOYED ON SEGMENTATION MAPS OBTAINED VIA THE UNET [46]

Method	Dice↑	Dice_l ↑	β_0↓	β_1↓	Xerror↓
UNet (baseline)	0.816	0.810	0.792	0.327	1.068
Closing (kernel size = 3)	0.814	0.810	0.704	0.333	0.984
Closing (kernel size = 5)	0.803	0.800	0.587	0.356	0.871
Closing (kernel size = 7)	0.782	0.781	0.525	0.383	0.826
ConvCRF (kernel size = 1)	0.775	0.771	0.752	0.371	1.052
ConvCRF (kernel size = 3)	0.706	0.702	0.718	0.398	1.039
ConvCRF (kernel size = 5)	0.691	0.688	0.711	0.404	1.039
CascadePSP	0.641	0.448	1.133	0.502	1.583
SegRefiner	0.790	0.785	0.599	0.346	0.891
Deep Closing (ours)	0.816	**0.813**	**0.509**	**0.314**	**0.756**

nected regions. We have therefore rigorously calibrated this hyperparameter, presenting its varied performance metrics.

As the kernel size increases, both the closing operation and ConvCRF demonstrate the ability to connect small disconnected regions, as evidenced by the reduction in β_0. Concurrently, there is a diminishing of loops or holes, indicated by the increase in β_1. In contrast, general post-processing methods like CascadePSP and SegRefiner show suboptimal performance in vessel segmentation. These methods, typically designed for segmenting larger and more distinct objects, struggle with the subtlety and complexity required for accurately delineating the fine structures in vascular networks. Given that these deep learning-based methods heavily rely on training data features, they may not adequately address the issue of high variance and low contrast prevalent in medical images, particularly in capturing vascular structures.

Most notably, while other post-processing methods substantially lowered overlap metrics with minimal topological improvements, our method struck a superior balance between overlap accuracy and topological integrity. This achievement stems from our method's proficiency in learning the shape priors of tubular structures. Such learning allows for a focused rectification of potential topological disconnects while minimizing alterations in topologically non-critical areas.

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on August 14,2026 at 07:34:55 UTC from IEEE Xplore. Restrictions apply.

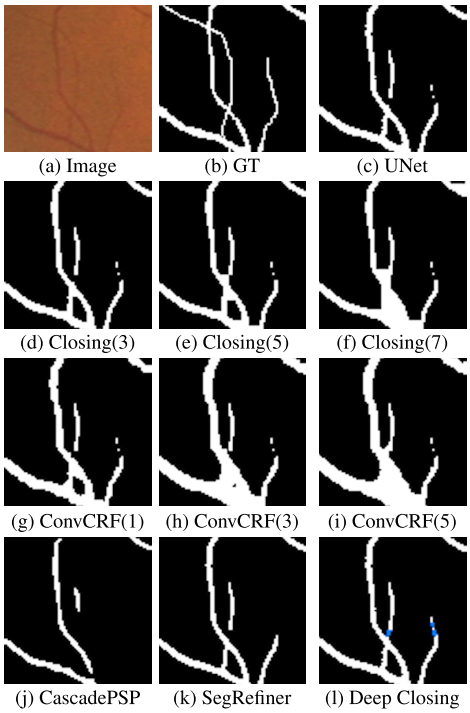


Fig. 5. Qualitative comparison with other post-processing methods, including classic Closing operation, ConvCRF [15], CascadePSP [16] and SegRefiner [17]. The numbers inside the parentheses indicate the sizes of the kernel used for Closing and ConvCRF operations.

Our method not only upholds the structural integrity of complex vascular networks but also excels in maintaining their unique topological features. This dual emphasis on structural preservation and topological precision vividly demonstrates the efficacy of our approach in meeting the specific segmentation challenges associated with tubular structures.

F. Transferrability

In order to assess the real-world applicability and showcase the efficacy of the acquired tubular shape prior through our proposed methods, we have devised a series of transfer experiments involving the extraction of shape priors from satellite road maps. This dataset is referred to as the **Road** dataset.

The **Road** dataset, sourced from the Massachusetts Roads dataset [47], comprises a collection of 1108 images, each accompanied by its corresponding road map annotations. These images possess a resolution of 1500 × 1500, with the road annotations provided at the same scale, including the specification of thickness.

By employing the Deep Closing (DC) model pre-trained on the **Road** dataset, we have applied it to the UNet-presegmented results across four datasets. The outcomes of these experiments are summarized in Table III. A pattern similar to that observed in IV-D becomes apparent, underscoring the model's ability to generalize and transfer the acquired shape prior effectively. Furthermore, to emulate a scenario devoid of labeled data, we have also employed the pre-trained model in a self-supervised approach [48] on the DRIVE dataset, denoted as DRIVE(SelfSup). This application demonstrates that the DC model can enhance performance even when annotations are scarce or absent, further highlighting the practical utility of our approach.

TABLE III
RESULTS OF TRANSFERABILITY EXPERIMENTS. ALL THE DEEP CLOSING MODELS ARE TRAINED ON THE SATELLITE DATASETS **ROAD** [47]. THE METHOD "SELFSUP" REFERS TO A SELF-SUPERVISED TECHNIQUE [48]

Dataset	Dice↑	Dice_l ↑	β_0↓	β_1↓	Xerror↓
DRIVE	**0.816**	0.810	0.792	0.327	1.068
+ ours	0.812	**0.811**	**0.382**	**0.292**	**0.613**
DRIVE(SelfSup)	**0.707**	0.683	1.023	0.433	1.397
+ ours	0.705	**0.705**	**0.602**	**0.395**	**0.912**
CHASE_DB1	**0.826**	0.791	0.400	0.069	0.454
+ ours	0.825	**0.800**	**0.295**	**0.067**	**0.347**
DCA	0.784	0.761	0.358	0.068	0.421
+ ours	0.784	**0.786**	**0.201**	**0.064**	**0.265**
CREMI	0.914	0.910	0.093	0.132	0.215
+ ours	0.914	0.910	**0.091**	**0.131**	**0.213**

TABLE IV
ABLATION STUDY OF THE KEY COMPONENTS

Method	Dice↑	Dice_l ↑	β_0↓	β_1↓	Xerror↓
Deep Closing	0.816	0.813	0.509	0.314	0.756
w/o DynMask	0.815	0.810	0.704	0.327	0.979
w/o TopoLoss	0.815	0.810	0.610	0.310	0.880
w/o SCE	0.816	0.813	0.512	0.314	0.758
DC from Road	0.812	0.811	0.382	0.292	0.613
w/o TubeScaler	0.693	0.729	0.653	0.384	0.865

G. Ablation Study

To evaluate each module, we also conduct a series of ablation studies of the proposed method by removing some of the components or tuning key hyperparameters.

1) Effectiveness of Each Module: We first display the ablation study of components. including Dynamic Mask Generator (DynMask), Topological constraints during training (TopoLoss), TubeScaler, and Simple Component Erosion (SCE). The results are shown in Table IV. It could be observed that all the proposed components contribute to the performance. Note that the SCE module seems to bring very marginal improvement from the quantitative results in the table. It is because the SCE itself only affects a small proportion of pixels generated by the preceding deep dilation process, around 21% on the DRIVE dataset. Nevertheless, we argue that this module is of great importance as it could help filter out topologically unimportant generated regions. This enhancement is crucial not only for elevating the visual clarity and interpretability of segmentation results but also for addressing the intricate details of small-scale structures, thereby improving the method's practical utility in detailed analysis, e.g., at the patch-level. As shown in Fig. 6, tiny and topologically simple components are eroded while only those bridge two or more regions are preserved.

2) Mask Ratio and Patch Size: The careful selection of the mask ratio and patch size holds paramount importance, as they play a pivotal role in determining the efficacy of the trained model. Defining an appropriate range for these two hyperparameters entails an exhaustive exploration through

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on August 14,2026 at 07:34:55 UTC from IEEE Xplore. Restrictions apply.

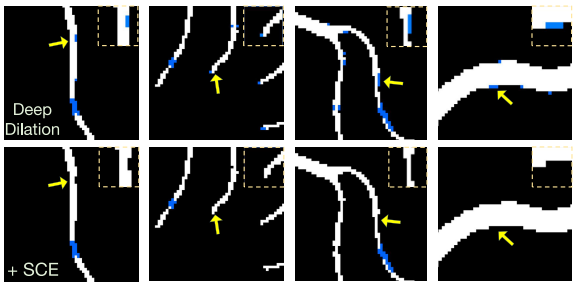


Fig. 6. Qualitative Evaluation of the proposed Simple Component Erosion (SCE) Module. The top row showcases the results following the Deep Dilation process, while the bottom row displays the outcomes subsequent to the application of the SCE module. Noteworthy altered areas are highlighted using yellow indicators.

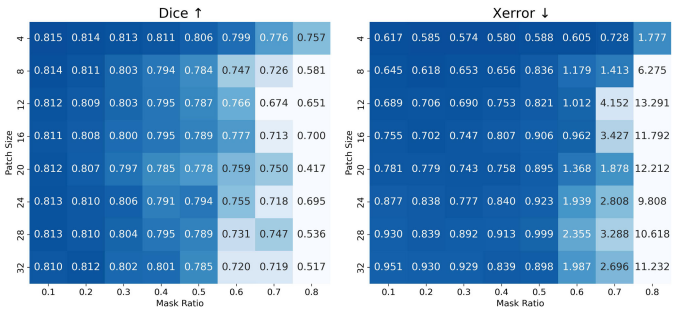


Fig. 7. A grid search of mask ratio and patch size.

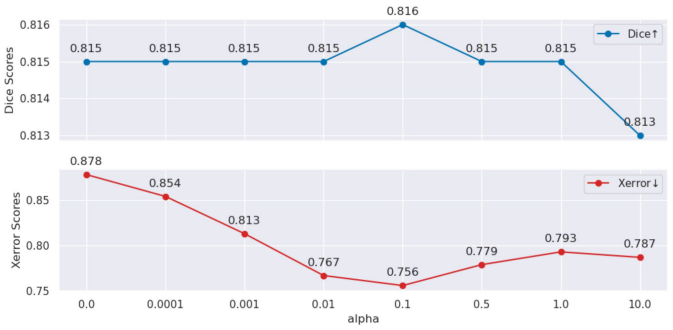


Fig. 8. The selection of alpha for $Loss_{topo}$.

TABLE V
RUNNING TIME ANALYSIS OF THE PROPOSED TOPOLOGICAL OPERATION PSPC, AND ITS COMPARISON WITH [7]

Method	Time Complexity	Running Time (s) Under n pixels				
		$n=10^2$	100^2	500^2	1000^2	10000^2
Hu [7] (CPU)	O(n)	6e-3	0.5	11	45	4.4e3
Ours (CPU)	O(n)	2e-4	2e-4	2e-3	1e-2	1
Ours (GPU)	O(n)	2e-4	2e-4	2e-4	2e-4	2e-2

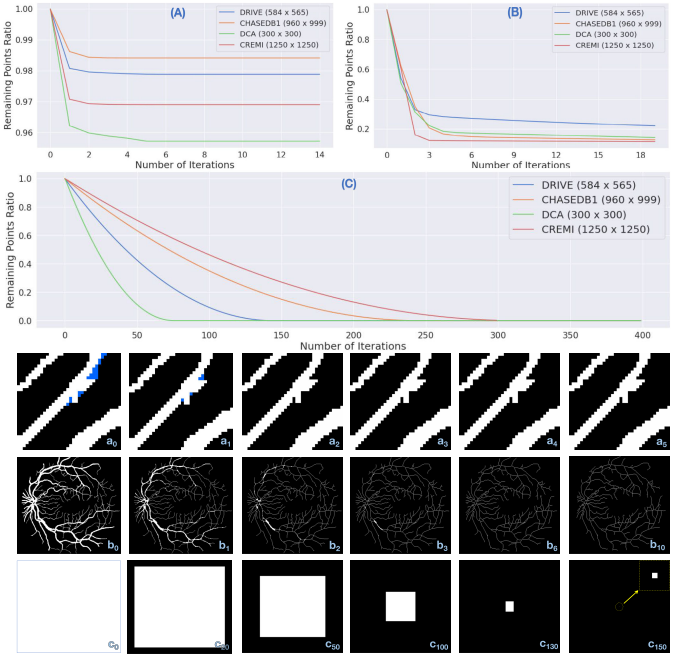


Fig. 9. Exploring the proper iteration number k for the Recurrent Simple Point Erosion (RSPE) module under different scenarios. Curve (A) illustrates RSPE's capability to eliminate topologically insignificant points (simple points) from specific structures. This process is visualized through the sequence $a_0 \sim a_{10}$, highlighting the gradual erosion of target points (depicted in blue) over iterations. Curve (B) reveals how RSPE tunes the scale of tubular structures, with the procedure depicted in the sequence $b_0 \sim b_{10}$. Curve (C) displays the erosion process in extreme scenarios (using a fully marked image), demonstrated in the series $c_0 \sim c_{300}$. The i in a_i, b_i, c_i stands for the iteration number of *RSPE*.

an 8 * 8 grid search, where we leverage the $Dice$ coefficient and the X_{error} metric as indicators for segmentation and topological quality, respectively. The outcomes of this search are visually represented in Fig. 7.

Upon analyzing the results of the grid search, a compelling pattern emerges. It becomes evident that maintaining a mask ratio within the range of 0.1 to 0.2, coupled with a patch size between 4 to 8, empowers the Deep Closing model to excel in both segmentation accuracy and topological performance.

3) Alpha for TopoLoss: In Fig. 8, we display the model performance over α, which is the weight of topological constraints, as defined in Eq. 4. It could be observed that when α is around 0.1, the model could achieve better results.

4) Efficiency Analysis of the Proposed Topological Operations: In Table V, we provide a running time analysis of the proposed topological operation **PSPC** under inputs with diversified sizes, to showcase the efficiency of our method. Since the only iteration in this operation is to slidingly extract 3×3 patches, and the time complexity of indexing value from the lookup table could be considered a constant, our approach achieves a time complexity of $O(n)$. It can be observed that our method has already demonstrated significant performance superiority when tested on CPUs. With the strong and parallel computation capability powered by GPUs, our method further achieves unprecedented efficiency, which even reaches an acceleration ratio around $2*10^5$ under $n = 10000^2$ (compared with Hu et al. [7]). Additionally, the acceleration could further increase as n grows or when tested with a large batch of inputs, enhancing the feasibility of integrating our method into deep-learning model pipelines.

5) How to Determine the Proper Number of Iterations k for RSPE: Recall that the RSPE module plays two roles in the proposed framework:

- *Function1:* Identifying topologically critical points for computing the topological loss $Loss_{topo}$ and removing topologically unimportant components (simple components) during the inference stage.
- *Function2:* Generating diversified scales of tubular structures as an effective data augmentation method,

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on August 14,2026 at 07:34:55 UTC from IEEE Xplore. Restrictions apply.

driven by the RSPE operation through the designed Recurrent TubeScaler.

We investigate the proper number of iterations, k, across four datasets to comprehend its influence on two specific functions. For *function 1*, the effect of consecutively applying the RSPE module to the selected samples from the DRIVE, CHASEDB, DCA, and CREMI datasets is analyzed. The ratio of remaining pixels over iterations is depicted, and is presented as curve (A) in Figure 9. This demonstrates that the RSPE module efficiently pinpoints critical points, typically within 5 steps. In the case of *function 2*, which entails adjusting the scale of tubular structures, our analysis suggests an optimal k range of $0 \sim 4$, as demonstrated by curve (B) in the same figure. A boundary testing with full-one images, which represent extreme scenarios, reveals that the number of required iterations correlates with image size but remains effective; for instance, 300 iterations on a 1250×1250 image are completed in approximately 0.5 seconds on our system. The visual evolution of these processes is illustrated in sequences $a_0 \sim a_{10}$ for *function 1* and $b_0 \sim b_{10}$ for *function 2*, with extreme cases depicted in $c_0 \sim c_{300}$. To conclude, the optimal choice of k is context-dependent: for *function 1*, a higher k (e.g., 10) may be required to accurately identify critical points in structures. Moreover, users can determine the RSPE's convergence state by checking if any point has been eroded in the latest iteration. For *function 2*, a narrower range of $0 \sim 4$ can be sufficient for most scenarios.

V. Conclusion

We propose a post-processing method that effectively identifies potential disconnected regions in tubular structures. While end-to-end approaches are currently favored, we believe our method holds significant value for the following reasons: (1) Our method has the potential to further improve the topological performance of existing end-to-end methods. (2) In situations where privacy concerns or limited data accessibility prevent access to the original data, our method can produce enhanced outcomes via analyzing shape information alone. (3) Unlike end-to-end approaches, our method generates new topologically critical regions without altering the original output. This makes it more explainable and suitable for interactive diagnosis. (4) Even when both image and label data are inaccessible, our method can transfer the learned inherent shape prior from other datasets to the target field.

In this paper, we introduce a novel framework that redefines traditional morphological closing operations through the integration of advanced deep learning techniques. Our approach utilizes the reconstruction capabilities of a trained masked AutoEncoder, which we term *Deep Dilation*, to effectively highlight areas of disconnection. Additionally, we have developed a module called *Simple Point Erosion*, aimed at identifying topologically critical points by selectively eroding non-essential ones (simple points). This further leads to the design of a novel operation, *Simple Component Erosion*, designed to eliminate topologically insignificant components within structures.

Our proposed framework comprises these two innovative operations. It demonstrates significant potential in identifying regions of disconnection, thereby enhancing the topological performance of medical tubular segmentation. Moreover, our method exhibits a strong capability to transfer shape priors from external datasets, showcasing its versatility and effectiveness across different applications.

References

[1] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in *Proc. 18th Int. Conf. Med. Image Comput. Comput.-Assist. Intervent.*, vol. 9351, Munich, Germany. Cham, Switzerland: Springer, Oct. 2015, pp. 234–241.

[2] F. Isensee, P. F. Jaeger, S. A. Kohl, J. Petersen, and K. H. Maier-Hein, "nnU-Net: A self-configuring method for deep learning-based biomedical image segmentation," *Nature Methods*, vol. 18, no. 2, pp. 203–211, 2021.

[3] R. Gharleghi, N. Chen, A. Sowmya, and S. Beier, "Towards automated coronary artery segmentation: A systematic review," *Comput. Methods Programs Biomed.*, vol. 225, Oct. 2022, Art. no. 107015.

[4] O. O. Sule, "A survey of deep learning for retinal blood vessel segmentation methods: Taxonomy, trends, challenges and future directions," *IEEE Access*, vol. 10, pp. 38202–38236, 2022.

[5] X. Hu, F. Li, D. Samaras, and C. Chen, "Topology-preserving deep image segmentation," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 1–12.

[6] X. Hu, Y. Wang, L. Fuxin, D. Samaras, and C. Chen, "Topology-aware segmentation using discrete Morse theory," in *Proc. Int. Conf. Learn. Represent.*, 2021, pp. 1–13.

[7] X. Hu, "Structure-aware image segmentation with homotopy warping," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 24046–24059.

[8] X. Hu, D. Samaras, and C. Chen, "Learning probabilistic topological representations using discrete Morse theory," 2022, *arXiv:2206.01742*.

[9] S. Shit et al., "ClDice—A novel topology-preserving loss function for tubular structure segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 16555–16564.

[10] D. Wang, Z. Zhang, Z. Zhao, Y. Liu, Y. Chen, and L. Wang, "PointScatter: Point set representation for tubular structure extraction," in *Proc. 17th Eur. Conf. Comput. Vis. (ECCV)*, Tel Aviv, Israel. Berlin, Germany: Springer, 2022, pp. 366–383, doi: 10.1007/978-3-031-19803-8_22.

[11] Z. Yang and S. Farsiu, "Directional connectivity-based segmentation of medical images," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 11525–11535.

[12] R. J. Araújo, J. S. Cardoso, and H. P. Oliveira, "A deep learning design for improving topology coherence in blood vessel segmentation," in *Medical Image Computing and Computer Assisted Intervention—MICCAI*, D. Shen, T. Liu, T. M. Peters, L. H. Staib, C. Essert, S. Zhou, P.-T. Yap, and A. Khan, Eds. Cham, Switzerland: Springer, 2019, pp. 93–101.

[13] D. Jha, M. A. Riegler, D. Johansen, P. Halvorsen, and H. D. Johansen, "DoubleU-Net: A deep convolutional neural network for medical image segmentation," in *Proc. IEEE 33rd Int. Symp. Comput.-Based Med. Syst. (CBMS)*, Jul. 2020, pp. 558–564.

[14] N. Tajbakhsh, B. Lai, S. P. Ananth, and X. Ding, "ErrorNet: Learning error representations from limited data to improve vascular segmentation," in *Proc. IEEE 17th Int. Symp. Biomed. Imag. (ISBI)*, Apr. 2020, pp. 1364–1368.

[15] M. Teichmann and R. Cipolla, "Convolutional CRFs for semantic segmentation," in *Proc. Brit. Mach. Vis. Conf.*, 2018, pp. 1–13. [Online]. Available: https://api.semanticscholar.org/CorpusID:21655707

[16] H. K. Cheng, J. Chung, Y.-W. Tai, and C.-K. Tang, "CascadePSP: Toward class-agnostic and very high-resolution segmentation via global and local refinement," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 8890–8899.

[17] M. Wang, H. Ding, J. H. Liew, J. Liu, Y. Zhao, and Y. Wei, "SegRefiner: Towards model-agnostic segmentation refinement with discrete diffusion process," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 36, 2024, pp. 1–13.

[18] Z. Zhou, M. M. R. Siddiquee, N. Tajbakhsh, and J. Liang, "UNet++: Redesigning skip connections to exploit multiscale features in image segmentation," *IEEE Trans. Med. Imag.*, vol. 39, no. 6, pp. 1856–1867, Jun. 2020.

[19] Q. Wu, Y. Chen, N. Huang, and X. Yue, "Weakly-supervised cerebrovascular segmentation network with shape prior and model indicator," in *Proc. Int. Conf. Multimedia Retr.*, 2022, pp. 668–676.

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on August 14,2026 at 07:34:55 UTC from IEEE Xplore. Restrictions apply.

[20] Z. Gu et al., "CE-Net: Context encoder network for 2D medical image segmentation," *IEEE Trans. Med. Imag.*, vol. 38, no. 10, pp. 2281–2292, Oct. 2019.

[21] H. Fu, Y. Xu, S. Lin, D. W. K. Wong, and J. Liu, "DeepVessel: Retinal vessel segmentation via deep learning and conditional random field," in *Medical Image Computing and Computer-Assisted Intervention—MICCAI*, S. Ourselin, L. Joskowicz, M. R. Sabuncu, G. Unal, and W. Wells, Eds. Cham, Switzerland: Springer, 2016, pp. 132–139.

[22] N. Stucki, J. C. Paetzold, S. Shit, B. Menze, and U. Bauer, "Topologically faithful image segmentation via induced matching of persistence barcodes," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 32698–32727.

[23] S. Gupta, Y. Zhang, X. Hu, P. Prasanna, and C. Chen, "Topology-aware uncertainty for image segmentation," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 36, 2024, pp. 1–14.

[24] X. Han, C. Xu, and J. L. Prince, "A topology preserving level set method for geometric deformable models," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 25, no. 6, pp. 755–768, Jun. 2003.

[25] C. Le Guyader and L. A. Vese, "Self-repelling snakes for topology-preserving segmentation models," *IEEE Trans. Image Process.*, vol. 17, no. 5, pp. 767–779, Mar. 2008.

[26] G. Sundaramoorthi and A. Yezzi, "Global regularizing flows with topology preservation for active contours and polygons," *IEEE Trans. Image Process.*, vol. 16, no. 3, pp. 803–812, Feb. 2007.

[27] P. Wu et al., "Optimal topological cycles and their application in cardiac trabeculae restoration," in *Information Processing in Medical Imaging*, M. Niethammer, M. Styner, S. Aylward, H. Zhu, I. Oguz, P.-T. Yap, and D. Shen, Eds. Cham, Switzerland: Springer, 2017, pp. 80–92.

[28] C. Sutton and A. McCallum, "An introduction to conditional random fields," *Found. Trends Mach. Learn.*, vol. 4, no. 4, pp. 267–373, 2012.

[29] H. Bao, L. Dong, S. Piao, and F. Wei, "BEIT: BERT pre-training of image transformers," in *Proc. Int. Conf. Learn. Represent.*, Jun. 2021, pp. 1–13.

[30] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, "Masked autoencoders are scalable vision learners," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 16000–16009.

[31] Z. Xie et al., "SimMIM: A simple framework for masked image modeling," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 9653–9663.

[32] A. Dosovitskiy et al., "An image is worth 16×16 words: Transformers for image recognition at scale," 2020, *arXiv:2010.11929*.

[33] Z. Liu et al., "Swin Transformer: Hierarchical vision transformer using shifted windows," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 10012–10022.

[34] T. Y. Kong and A. Rosenfeld, "Digital topology: Introduction and survey," *Comput. Vision, Graph., Image Process.*, vol. 48, no. 3, pp. 357–393, 1989.

[35] J. Staal, M. D. Abràmoff, M. Niemeijer, M. A. Viergever, and B. Van Ginneken, "Ridge-based vessel segmentation in color images of the retina," *IEEE Trans. Med. Imag.*, vol. 23, no. 4, pp. 501–509, Apr. 2004.

[36] W. Liu et al., "Full-resolution network and dual-threshold iteration for retinal vessel and coronary angiograph segmentation," *IEEE J. Biomed. Health Informat.*, vol. 26, no. 9, pp. 4623–4634, Sep. 2022.

[37] M. M. Fraz et al., "An ensemble classification-based approach applied to retinal blood vessel segmentation," *IEEE Trans. Biomed. Eng.*, vol. 59, no. 9, pp. 2538–2548, Sep. 2012.

[38] F. Cervantes-Sanchez, I. Cruz-Aceves, A. Hernandez-Aguirre, M. A. Hernandez-Gonzalez, and S. E. Solorio-Meza, "Automatic segmentation of coronary arteries in X-ray angiograms using multiscale analysis and artificial neural networks," *Appl. Sci.*, vol. 9, no. 24, p. 5507, Dec. 2019.

[39] J. Yang, X. Hu, C. Chen, and C. Tsai, "A topological-attention ConvLSTM network and its application to EM images," in *Medical Image Computing and Computer Assisted Intervention—MICCAI*, M. de Bruijne, P. C. Cattin, S. Cotin, N. Padoy, S. Speidel, Y. Zheng, and C. Essert, Eds. Cham, Switzerland: Springer, 2021, pp. 217–228.

[40] I. Arganda-Carreras et al., "Crowdsourcing the creation of image segmentation algorithms for connectomics," *Frontiers Neuroanatomy*, vol. 9, p. 142, Nov. 2015.

[41] M. Meilă, "Comparing clusterings—An information based distance," *J. Multivariate Anal.*, vol. 98, no. 5, pp. 873–895, May 2007.

[42] S. van der Walt et al., "Scikit-image: Image processing in Python," *PeerJ*, vol. 2, p. e453, Jun. 2014.

[43] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 1–12.

[44] M. Jorge Cardoso et al., "MONAI: An open-source framework for deep learning in healthcare," 2022, *arXiv:2211.02701*.

[45] T. Shi, N. Boutry, Y. Xu, and T. Géraud, "Local intensity order transformation for robust curvilinear object segmentation," *IEEE Trans. Image Process.*, vol. 31, pp. 2557–2569, 2022.

[46] Ö. Çiçek, A. Abdulkadir, S. S. Lienkamp, T. Brox, and O. Ronneberger, "3D U-Net: Learning dense volumetric segmentation from sparse annotation," in *Medical Image Computing and Computer-Assisted Intervention—MICCAI*, S. Ourselin, L. Joskowicz, M. R. Sabuncu, G. Unal, and W. Wells, Eds. Cham, Switzerland: Springer, 2016, pp. 424–432.

[47] V. Mnih, *Machine Learning for Aerial Image Labeling*. Toronto, ON, Canada: Univ. Toronto, 2013.

[48] R. Jena, S. Singla, and K. Batmanghelich, "Self-supervised vessel enhancement using flow-based consistencies," in *Medical Image Computing and Computer Assisted Intervention—MICCAI*, M. de Bruijne, P. C. Cattin, S. Cotin, N. Padoy, S. Speidel, and Y. Zheng, Eds. Cham, Switzerland: Springer, 2021, pp. 242–251.

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on August 14,2026 at 07:34:55 UTC from IEEE Xplore. Restrictions apply.